PM2.5 One-Hour-Ahead Forecasting

Model design report -- feature engineering, model choice, and hyperparameter rationale

1. Task and Data

The task is to predict PM2.5 concentration one hour ahead, for a given monitoring station and hour, using that hour's own readings and the station's history. **train.csv** provides hourly readings for 12 Beijing monitoring stations, each row carrying the current hour's pollutant levels (PM2.5, PM10, SO2, NO2, CO, O3), weather (temperature, pressure, dew point, rainfall, wind direction and speed), and the label PM2_5_next_hour. **test.csv** has the same feature columns, without the label.

Every row in both files already carries its own real, sensor-measured "current hour" readings (a small fraction, under 1%, has a missing current PM2.5 reading due to sensor gaps, handled explicitly below). This means the task is a repeated one-step-ahead forecast, not a multi-step recursive forecast: the model never has to feed its own earlier predictions back in as input for a later prediction, so there is no error accumulation across hours.

Exploratory analysis of train.csv shows two data characteristics that shaped every design decision below:

- **Strong seasonality.** PM2.5 levels and hour-to-hour volatility are far higher in the autumn/winter heating season than in spring/summer.
- **Heavy-tailed, spiky behaviour.** Even within a single winter month, PM2.5 can swing from single digits to several hundred micrograms per cubic metre within days, driven by weather-dependent pollution build-up and clearance episodes.

2. Validation Strategy

This is a time series, so a random train/validation split is not appropriate: it would let information from a given week leak into both sides of the split via lag and rolling-window features computed from neighbouring rows, and it would not test the model's ability to generalise to a genuinely future, unseen period the way the real train/test split does.

All model selection below (hyperparameters, boosting rounds, ensemble weight) instead uses a **chronological** validation scheme with two folds, each holding out several months that come after the corresponding training cut-off:

Fold	Trained on	Validated on
A	data up to a late-summer cut-off	the following autumn/winter block
B	data up to the following late-winter cut-off	the subsequent spring/summer block

Fold A's validation window is deliberately the same season mix (autumn through winter) as the eventual test period, making it the more representative of the two folds; Fold B checks that the model also generalises to a different season rather than only the one it was tuned for.

3. Feature Engineering

Complete hourly time grid. Each station's rows are re-indexed onto a full hourly timeline before any lag or rolling feature is computed. Without this step, a station with a gap in its readings would have its "1 hour ago" lag silently point to whatever row happens to precede it in the raw file, which could actually be several hours earlier -- the grid guarantees that a lag of k always means exactly k hours, and short gaps (up to 24h) are forward-filled rather than left as unexplained missing blocks.

Lag and rolling-window features (1-24 hours). PM2.5 is strongly autocorrelated hour to hour, so recent history -- lagged values, rolling mean/std/max, and the deviation of the current reading from its recent rolling mean -- is the single most informative feature family. The same lag/rolling treatment is applied to the other pollutants and weather variables, since they co-move with PM2.5 on similar time scales.

Wind vector decomposition. Wind direction is provided as a 16-point compass label. Converting direction and speed into orthogonal u/v components (rather than keeping a categorical bearing) gives the model a representation where similar wind conditions are numerically close and opposite conditions are numerically far apart, which a raw compass label does not provide.

Cyclical calendar encoding. Hour-of-day and day-of-year are encoded as sine/cosine pairs rather than raw integers, so the model sees hour 23 and hour 0 as adjacent, and December 31st and January 1st as adjacent, instead of as maximally distant values.

City-wide aggregate. The mean, standard deviation and maximum PM2.5 across all 12 stations at the same hour capture city-scale pollution episodes (e.g. a stagnant-air event) that affect every station together, not just the one being predicted.

3.1 Cross-station "donor" feature

The city-wide average above is isotropic: it weights every other station equally, regardless of whether pollution is actually likely to travel from that station to the one being predicted. Since the raw data does not include station coordinates, direct wind-transport geometry cannot be computed. Instead, for each station, the training data is used to identify the two or three *other* stations whose current PM2.5 is most strongly correlated with this station's next-hour PM2.5. A correlation-weighted average of those "donor" stations' current readings is added as a new feature.

This is a legitimate, leakage-free feature: it uses only same-timestamp readings from other stations, which are already known at prediction time for every row (the same principle already used by the city-wide aggregate above), and the donor relationships themselves are learned from the training data only. The resulting station groupings recover a plausible geographic structure purely from co-movement in the data -- stations known to sit close together in central Beijing consistently pick each other as donors, and outlying stations form a separate group -- without ever being given location information.

A time-lagged version of this feature (using the donor stations' readings from 1-3 hours earlier, to capture pollution transport delay, plus the resulting trend) was also tried, since pollution transport between locations is not instantaneous. It was not included in the final model: validation performance was consistently worse than the same-hour-only version, most likely because the added lagged columns were highly redundant with each other and with the target station's own trend features, diluting rather than sharpening the tree-building process.

4. Target Transformation

Rather than predicting the next hour's PM2.5 level directly, the model predicts the **change** (PM2_5_next_hour minus the current reading), and the final forecast is recovered by adding that predicted change back to the current reading. Current PM2.5 is by a wide margin the single strongest predictor of next-hour PM2.5; modelling the change directly removes this dominant, easy component from the target and lets the model's capacity focus on the harder part of the problem -- how conditions are about to shift -- rather than spending most of its effort re-deriving a value that is already known.

When the current hour's own PM2.5 reading is missing (a rare sensor gap, under 1% of rows), the most recent available reading for that station is used in its place for this reconstruction step.

5. Model Choice

5.1 LightGBM (primary model)

The feature set is tabular: a large number of numeric lag/rolling/weather features plus one categorical feature (station), with meaningful non-linear interactions expected between them (for example, the effect of wind speed on PM2.5 plausibly depends on wind direction and season together). Gradient-boosted trees are a strong default for this kind of data: they model non-linear interactions automatically without manual interaction terms, need no feature scaling, and handle the missing values produced by sensor gaps natively rather than requiring a separate imputation step.

Huber loss is used instead of squared error. Squared error weights large residuals quadratically, so the rare but large pollution spikes noted in Section 1 would dominate the loss and pull the fit away from typical conditions in order to chase a handful of extreme points. Huber loss behaves like squared error for small residuals but like absolute error beyond a threshold, giving a fit that is far less distorted by outlier spikes while still being sensitive to them.

Alternative model families were also evaluated under the same two-fold validation scheme, including other gradient-boosting implementations, a random-forest-style tree ensemble, a feed-forward neural network, a linear model, and a classical autoregressive time-series model fitted per station. LightGBM with the feature set and target transformation above was the strongest single model in every case.

5.2 GRU sequence model (ensemble partner)

A second model -- a GRU (gated recurrent unit) reading the raw last 24 hours of a station's readings directly, with a learned per-station embedding -- is trained and blended with LightGBM's predictions. On its own, the GRU is less accurate than LightGBM. It is still worth including in the final forecast because the two models make *different kinds* of errors: LightGBM reasons over hand-built summary features, while the GRU learns its own representation directly from the raw sequence. Averaging two models whose errors are only partially related reduces the combined error below either model's own error, provided the weaker model is not so much weaker that its own noise outweighs the diversity benefit -- which is why a small blend weight, rather than an equal one, is used.

Other candidate ensemble partners (a second gradient-boosting model on the same feature set, a linear model, the per-station time-series model) were also evaluated as blend partners under the same validation scheme. Models sharing LightGBM's tree-based, same-feature-set structure consistently produced errors too similar to LightGBM's own to add meaningful ensemble benefit; the GRU's fundamentally different

architecture and input representation made it the most useful complement.

6. Hyperparameter Selection

LightGBM's hyperparameters were selected with an automated Bayesian search (Optuna, using a Tree-structured Parzen Estimator sampler) over learning rate, tree complexity (num_leaves, max_depth, min_data_in_leaf), row/feature subsampling fractions, L1/L2 regularisation, and the Huber loss threshold, each trial scored by the two-fold validation scheme in Section 2. The selected configuration:

Hyperparameter	Value	Role
learning_rate	0.0211	step size per boosting round
num_leaves	91	per-tree complexity
max_depth	11	caps tree depth as a secondary complexity control
min_data_in_leaf	13	minimum samples per leaf, limits overfitting on rare patterns
feature_fraction	0.999	fraction of features sampled per tree
bagging_fraction	0.775	fraction of rows sampled per iteration
lambda_l1 / lambda_l2	0.062 / 0.142	L1 / L2 leaf-weight regularisation
huber alpha (delta)	65.7	residual threshold where the loss switches from quadratic to linear

Number of boosting rounds. A single early-stopping run on one validation fold can stop too early or too late depending on the noise in that particular fold, and is not guaranteed to match the round count that performs best once the model is refit on the full training data. The final round count, 3800, was instead chosen by tracking validation performance across a wide range of round counts and selecting the point of best average performance across both folds, rather than trusting a single early-stopping run.

Five-seed averaging. LightGBM's row and feature subsampling is stochastic, so five models differing only in random seed are trained and their predictions averaged. Validation performance was tracked as seeds were added one at a time; most of the benefit was realised within the first several seeds, with returns beyond that point becoming inconsistent, so five was used as a reasonable balance of benefit against training cost.

GRU blend weight. The weight given to the GRU's prediction in the final average (0.17, i.e. 83% LightGBM / 17% GRU) was chosen by sweeping the blend weight and selecting the value that minimised validation error, following the same reasoning as Section 5.2: enough weight to capture the GRU's complementary errors, not so much that its larger individual error dominates the blend.

7. Final Pipeline

- **feature_engineering.py** -- builds the lag/rolling/wind/calendar/city-wide/donor-station feature set described in Section 3, shared by both models.
- **train_lightgbm.py** -- trains 5 LightGBM models (different seeds) on the delta target with Huber loss, 3800 rounds each, and averages their predictions.
- **train_gru.py** -- trains the GRU sequence model on the same delta target.

- **run_all.py** -- runs both training scripts and combines their outputs as 0.83 × LightGBM + 0.17 × GRU, clipped to be non-negative, saved as **submission.csv**.

The LightGBM and GRU stages are run as separate processes rather than imported into one script: LightGBM and PyTorch each bring their own multi-threaded math library, and loading both into a single Python process was found to occasionally stall LightGBM's thread pool rather than raise a visible error. Running them as independent processes avoids the conflict entirely.

8. Known Limitations

- **Missing current-hour readings.** Under 1% of test rows have no current-hour PM2.5 reading due to sensor gaps; these rows fall back to the station's most recent known reading. A dedicated model to predict the missing reading from other same-hour signals was tested and did not outperform this simple fallback on the rows that are actually missing, likely because sensor gaps are not random -- they appear to coincide with atypical, harder-to-predict conditions, which a model trained mostly on typical (non-missing) rows does not represent well.
- **Extreme pollution episodes remain the hardest cases.** Both the raw PM2.5 level and the model's error are largest during peak heating-season conditions (roughly December-January), and the model's relative improvement over a naive same-value forecast is smallest exactly during these periods. This appears to be an inherent property of sharp, weather-driven pollution spikes -- they are the least predictable events from historical patterns alone -- rather than a specific, fixable gap identified in the current feature set.

End of report.